

# SWE S2026 FAQs

## SoundCloud Specific:

### 1. How does the authentication system work?

- Users can register using an email and password.
- A CAPTCHA is required during registration to mitigate bot activity.
- An automated confirmation email is sent upon registration to verify the account.
- Login is supported via email/password or through Google social login.
- Account recovery is handled via password reset requests sent to the registered email.
- For Social login (Only one platform is enough i.e. Google , Facebook,..)
- 2FA not required

### 2. What information can users add to their profiles?

- Users can customize their display name, bio, and location.
- They can designate their account type as either an **Artist** (creator) or **Listener** (consumer).
- Users can upload a profile picture and a high-resolution cover photo.
- Profiles can be set to Public or Private to control visibility in search results.

### 3. How do followers and the social graph work?

- Users can follow/unfollow other profiles to see their latest uploads in their feed.
- The platform provides dedicated lists for "Followers" and "Following".
- Privacy is maintained through a blocking system, where users can manage a "Blocked Users" list.

### 4. How does the audio upload and management system work?

- Creators can upload audio in formats like MP3 and WAV.

- During upload, creators must provide metadata: Title, Genre, Description, and Tags.
- Tracks can be set to **Public** (searchable) or **Private** (accessible only via a unique link).
- The system validates track duration and generates a basic waveform visualization for the player.

## 5. How does audio playback and streaming work?

- Users can stream tracks with standard controls: Play, Pause, and Seek.
- The player tracks playback progress and maintains a "Recently Played" history for the user.
- The streaming player is designed to be persistent across the web and mobile interfaces.

## 6. How do social interactions on tracks work?

- Users can "Like" (favorite) or "Repost" tracks to their own profile feed.
- A comment system allows users to leave **timestamped comments** that appear at specific points during audio playback.
- Engagement metrics, such as total play counts and likes, are visible on each track.

## 7. How do playlists (Sets) work?

- Users can create, update, and delete custom playlists (often called "Sets").
- Tracks can be added, removed, or reordered within these playlists.
- Playlists can be shared via unique links and have their own privacy settings (Public/Private).

## 8. How does the Feed and Discovery system work?

- The **Activity Feed** displays new uploads, likes, and reposts from followed users in chronological order.
- A global search allows users to find tracks by title/tags or search for specific user profiles.
- A "Trending" section highlights popular tracks based on recent engagement and plays.

## 9. How does messaging work?

- Users can send one-to-one private text messages.
- Track and playlist links can be shared directly within these messages.
- Users can track unread message counts and block specific users from messaging them.

## 10. What notifications do users receive?

- Real-time notifications are triggered for new likes, comments, reposts, followers, and messages.
- Users can mark notifications as read and view an "unseen" notification count.

## 11. What is the role of the Admin Panel?

- Admins can monitor reports of inappropriate tracks or comments and remove them if necessary.
- The panel provides platform analytics, including total users, total tracks, and aggregate plays.

• So for the admin panel, you can generate any UI for it, preserving the analytics we want:

■ Total Users, Artist to Listeners Ratio

■ Total Tracks

■ Total Plays

■  $\text{Play Through Rate} = (\text{Total Plays} / \text{Completed Plays}) \times 100$   
Where: Completed Play = duration\_played >= 90% of track duration

■ Total Storage Used

## 12. How do the Premium plans and payments work?

- Payment processing is handled via **Stripe Test Mode** (mock payments).
- **Free Plan:**
  - Limited to 3 track uploads and 2 playlists.
  - Standard streaming access.
- **Premium Plan:**
  - Unlimited audio uploads and unlimited playlists.

- Mocked "Offline Listening" (downloading) capabilities.
- 

### Implementation Constraints (Course Specific)

- **Groups:** No group chats; only pairs chatting (Direct Messaging) is supported.
- **Reactions:** Standard "Likes" and timestamped comments are supported.
- **Replies:** Comment replies are restricted to **one level deep** only.
- **Message Reactions:** Not implemented.

### 13. What will be done in settings page?

- Basic account, privacy, content management (Provide the UI and functionality of RSS as is but no need to ensure it works with podcasts platform properly) , and notification toggles (Check Q11 for what is needed in Notifications).

### 14. The recommendation part is done by "stations" do they need to implement it as recommendation system ?

- Yes, but implement it as a simple rule-based discovery feature, NOT as an AI system.

#### Rule-Based Recommendation (Simple Logic)

#### Examples:

- Station by Genre → return tracks with same genre
- Station by Artist → return tracks from similar artists
- Trending Station → return most played in last X days
- Based on Likes → return tracks with shared tags

## General Project:

### 1. On what devices will cross-platform applications get tested?

You are free to use any cross-platform framework that can be compiled to run on any two different clients (Android/iOS, Android/Browser or Android/Desktop).

## **2. Can we change a tool after confirmation in phase 0?**

You should get the TA's confirmation, so provide a strong reason while making sure this won't affect other team members or the completion of the deliverables.

## **3. What is meant by "System Design" in phase 1?**

- Complete API documentation for the project.
- Main modules for the system (accompanied by all the features in each module). If you implement any feature differently from the original website (even in GUI), it should be clearly mentioned. This is very important, as this document will be used in the evaluation.
- The architecture and design patterns that will be used in the code.
  - o If not already mentioned in phase 0
- Naming conventions followed - database models used
- Any third party libraries or tools needed
- Github workflow (branch naming and the process of the PRs)

## **4. Who should attend the discussions?**

In Phase 0 and 1: The team leader and all sub-team leaders should attend.

Starting from Phase 2: All team members should attend the discussion.

## **5. What will happen to non-working members?**

1- If the team leader has reported that early enough, we can discuss the problem with the team member. If they continue in the same manner, he will be punished for up to 0 grades in the project. If the situation is repeated for the same member, the member might be kicked out from the team.

2- If the team leader did not report that, and we discovered that at the final phase of the project, the team leader, the sub-team leader, and the member would be severely penalized.

## **6. How should we demonstrate the project functionalities prototypes (50%, 100%)?**

There should be a seeder with logical data to be able to show all your functionalities. Make it as professional as possible. There should be another seed that contains only fundamental data, including types and one user, to run a clean version of your website.

## **7. What is the best way for authentication?**

Token-based authentication will be suitable for the project.

## **8. [DevOps] What exactly should we do in monitoring?**

You should monitor Resources usage [Internal Monitoring] and send an Email to the team leader when it exceeds some threshold. Besides monitoring the up-time and the internet disconnection and so on [External Monitoring].

## **9. Are we allowed to use any format for requests and responses?**

No, you should implement a RESTful API that sends and receives data in JSON format. You should implement the API using RESTful and CRUD concepts as illustrated in the tutorial. You can also check Internet resources for further info.

## **10. Is a general coding style for the language enough? For example, can I use a general javascript coding style?**

You need to be more specific. For example, use ReactJS coding style, not just javascript documentation.

## **11. Do we have to populate the database with the same data at the official API?**

No, you do not need to populate it with the same data in the official API. You need only to add the minimum data sufficient to make the application fully testable and running.

## **12. Is the backend team required to deliver Functional Documentation?**

Functional Documentation is required from BE, FE and Android/Cross platform teams.

## **13. Do we have to use oAuth like they are doing in the official API?**

No, you do not have to use the same method, Token-based authentication will be suitable for the project.

## **14. Is Docker required for the Backend Team?**

No, Docker is optional.

## **15. How are we supposed to write full API documentation before any coding?**

You already have the LinkedIn documentation as a reference, the endpoints you see relevant to the features we require in the project should be documented as soon as possible to ease integrating the work for the other sub-teams, later

modifications can be made but they should be kept to a minimum to avoid wasting time repeating work.

## **Integration:**

### **1. Who is responsible for integration?**

All the members are responsible for integrating the sub-team code ( for example, integrating the backend code is the responsibility of all the backend members)  
For the whole project, the sub-team leaders should handle the integration.

Note: The DevOps should be given clear instructions on building and integrating the front-end and the backend and how to execute the E2E testing. He is only responsible for automating the process, not understanding each detail of it!

### **2. What happens for failing to integrate the project?**

All the team members will be penalized, especially the team leader and the sub-leaders.